Flash Attention, Slowly

Starting from TinyGPT

SC26 nixnan tutorial

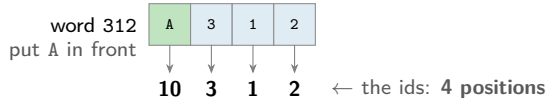
`flash-tutorial`

October 2026

Step 1: a word becomes a list of token ids



A (id 10) = start of a word
F (id 11) = end of a word



the same ids as one-hot rows (4×12)

	0	1	2	3	4	5	6	7	8	9	A	F
A	0	0	0	0	0	0	0	0	0	0	**1**	0
3	0	0	0	**1**	0	0	0	0	0	0	0	0
1	0	**1**	0	0	0	0	0	0	0	0	0	0
2	0	0	**1**	0	0	0	0	0	0	0	0	0

each row has a single 1, in the column of its id

- An id is a **label**: 3 is "token number 3", just as `A` is token number 10. The network never adds or compares ids as numbers.
- The next step turns each id into a vector of 128 numbers by picking one row of a table.

Step 2: each id picks one row of 128 numbers from the embedding table

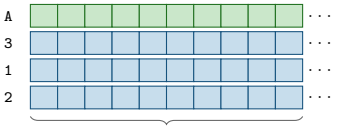
embedding table in `tinygpt.pt` (12×128, after training)

	col 0	col 1	col 2	col 3	⋯	col 127
0	−0.015	+0.010	−0.044	+0.061	⋯	+0.002
1	+0.037	−0.001	−0.059	−0.017	⋯	−0.139
2	+0.013	+0.025	+0.058	−0.007	⋯	+0.004
3	+0.033	+0.016	−0.141	+0.070	⋯	−0.077
4	−0.008	+0.048	+0.008	+0.039	⋯	−0.031
5	+0.030	+0.016	+0.027	+0.009	⋯	−0.013
6	+0.008	+0.066	+0.063	−0.054	⋯	+0.012
7	+0.075	+0.091	+0.013	−0.058	⋯	+0.014
8	+0.035	−0.010	+0.030	−0.076	⋯	+0.022
9	−0.014	−0.014	+0.020	+0.049	⋯	+0.078
A	−0.001	+0.000	−0.006	−0.012	⋯	−0.014
F	+0.010	+0.016	−0.020	+0.020	⋯	+0.005

12 rows: one per token

copy rows → A, 3, 1, 2

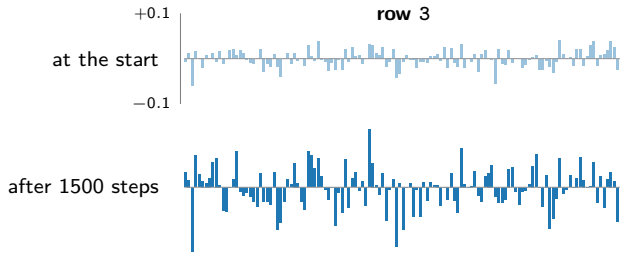
what block 0 gets for `A312`

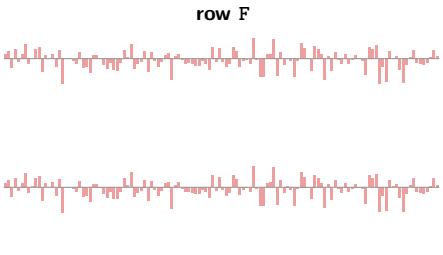


one-hot (4×12) × table (12×128) = these 4 rows (4×128): a matrix multiply that just copies rows

- Every entry is one **FP32 number** (32 bits). A row is 128 numbers, i.e. $128 \times 32 = 4096$ bits, not 128 bits.
- Row 3 is what the network means by 3. Its 128 numbers are coordinates of a point; no single column means anything by itself.

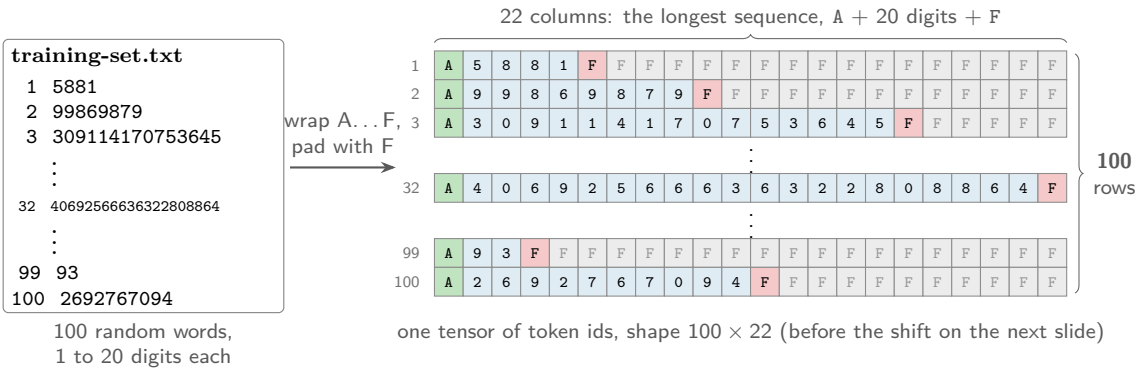
Where the numbers come from: random at first, then moved by training





- **Start:** every entry is drawn at random from a normal distribution with mean 0 and spread 0.02 (seed 0).
- **Training:** each of the 1500 AdamW steps nudges every entry in the direction that lowers the loss, exactly like TinyGPT's other parameters (the table is 1,536 of 791,808).
- **Why `F` never moves:** `F` is only ever an *input* after a word has ended, where the target is ignored (−100). Causal attention lets it affect only those positions, so its gradient is exactly zero.

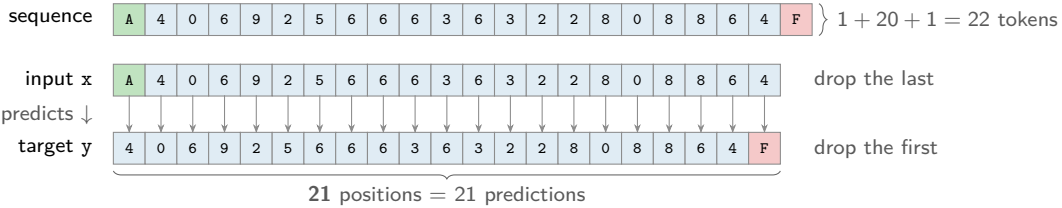
Where the batch size 100 comes from: one row per training word



There is no batch-size setting: `train.py` stacks *every* word of the training set into one batch and takes all 1500 optimizer steps on that same batch (no shuffling, no sampling).

Where 21 comes from: each position predicts the next token

word 32 (20 digits, the longest)

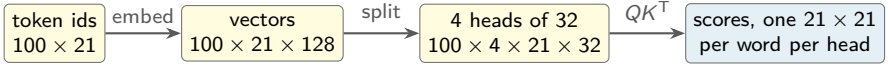


word 1 (5881): padded to the same 21

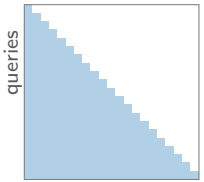


So the training batch is 100×21: 100 words, 21 positions each.

The batch on its way into attention

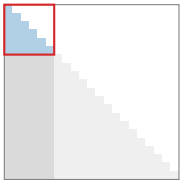


word 32: all 21 positions real



query i sees keys $0..i$ (causal)

word 1 (5881): 6 real, 15 padding



real queries see only real keys:
a pad comes *after* every real token,
so causal masking hides it.
No padding mask is needed.

pad queries (gray) are
computed, then ignored by the loss

Training (`train.py`): $100 \times 4 = 400$ score grids of 21×21 in each of the 4 blocks, every step, on the same batch for all 1500 steps.

Generation (`generate.py`): batch **1**. Prompt `A312` is one call ($1 \times 4 \times 4 \times 32$), then one call per new token, attending over the stored keys: up to 7 positions.